

...your fingers are required to carry out mouse and cursor operations from the keyboard itself.

Fully compatible with AT, PC, XT, PS/2 and Microsoft® mouse protocol.

Fatigue-free input as wrist can be rested. Changeable between mouse and cursor modes. Ideal for working with graphic user surfaces.

Through combination of both input systems, discrete backward and forward movement of hand between external mouse and keyboard is eliminated.

Send today for further technical details!

ARM600: RISC GOES OOP

be manipulated by particular programs, called the *methods* of the class to which the object belongs. It is not difficult to see how hardware-enforced client permissions can be used to protect objects from access by methods other than their own. Domains would be used to distinguish types of objects—in effect, a hardware expression of classes.

Many object-oriented programming researchers favor a secondary-storage scheme called a *persistent object store*. Objects become entities with their own

lives, the heap fills up with objects that are no longer being used, and the system would grind to a halt when so new objects could be created (or paged in) were it not for garbage collection. Traditional garbage-collection algorithms involve temporarily halting the system and scanning through memory to release space occupied by "dead" objects. You will be familiar with this annoying interruption if you've done serious programming in Lisp or Smalltalk.

The latest algorithms advocate the use of concurrent garbage collection. A garbage-collection manager runs as a background task, reclaiming dead objects as soon as they become inaccessible (i.e., when nothing else in the system points to them). This is preferable in interactive, real-time systems with GUIs, where a sudden freezing of the system would be most disconcerting.

One way of implementing concurrent garbage collection is to divide memory into a live region, where computation occurs, and a dead region, where garbage collection is continually proceeding. One technique is to make the live region a narrow address window that moves cyclically through the memory space, leaving a dead region behind it. The problem is that a few (if the algorithm is correctly tuned) live objects are likely to be left stranded in the dead region, still pointed to by objects in the live region. Such stranded objects have illegal addresses and must not be accessed (or even referred to).

A permission-mapping MMU like that in the ARM600 chip can prevent such accesses by raising a permission fault. To recover from such a fault, the operating system rescues the stranded object by copying it into the live region, where the requested access can proceed normally. The ARM600 chip's domain system suggests a separate, concurrent manager task to collect garbage in each domain. It makes possible the use of a second chip as a parallel garbage-collection processor.

Figure 2 shows the complete address-translation path for the ARM600 MMU and all its possible outcomes. Looking at it, you might wonder how any memory access ever succeeds. In practice, most accesses succeed immediately, from the TLB, and the system has been highly optimized to make the most likely cases the least time-consuming. The MMU can also reduce power consumption. Cycling the fast RAM of the TLB consumes most of the power. So, the address (as opposed to permission) segment of the TLB is

The ARM600 partitions memory on object-oriented lines.

life span, independent of the invocation of a particular application. When you are not using them, they live on disk, and when you are using them, they live in memory. The transition between these two states is transparent. If you need to access an object that is not in memory, this secondary-storage scheme brings the object into memory without your having to issue a load command or type in a filename. In fact, the concept of a file and the distinction between files and memory variables disappear altogether.

Some PC application systems, such as Borland's VRROOM, possess a persistent aspect (i.e., for code, not for data), but real persistence should be a property of the operating system itself (see "Tao: An Innovation in Operating Systems," March BYTE, International Section). The ARM600 MMU can offer hardware support for such a persistent object-oriented operating system by combining its virtual memory and permission-mapping roles. Such an operating system would be fault driven: The raising and subsequent correction of address, permission, and domain faults is what drives your computation.

There is more, however. In a persistent object-oriented operating system, garbage collection becomes an important issue. In any system that permits dynamic (i.e., run-time) allocation of mem-



Speed of system function programming is increased. Advantages for users: freeing the hand and saving precious space on desk and in bags.



Large store of mouse and cursor functions into the Cherry MousePad.



Activation of functions through gliding movements and pressing of integrated keys.

Circle 412 on Inquiry Card.

© 1991 by Cherry Mousepad GmbH.
Microsoft is the registered trademark of the Microsoft Corporation.